Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка веб-приложения»

студента IV курса 285 группы

Фамилия Шориков

Имя, отчество Антон Сергеевич

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 4
1 Учебная практика.....5
1.1 Анализ предметной области 5
1.2 Выбор стека технологий и инструментов разработки 6
1.3 Разработка базы данных.....7
1.4 Практическая реализация приложения.....9
1.5 Разработка веб-приложения.....12
1.6 Тестирование программы.....16
1.7 Качественная характеристика кода 17
2 Дипломный проект 18
2.1 Анализ предметной области 18
2.2 Разработка дополнительного функционала 19
Заключение 24
Библиография 25
Приложение А 26

Введение

Данная учебная практика направлена на подготовку к демонстрационному экзамену и описанию практической части дипломного проекта.

Цель учебной практики – проработать вариант демонстрационного экзамена и составить пояснительную записку к практической части дипломного проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать техническое задание и предметную область;
- сформировать базу данных и архитектуру программы;
- разработать приложение;
- разработать документацию к продукту;
- протестировать программу;
- проработать приложение дипломного проекта.

1 Учебная практика

1.1 Анализ предметной области

Разрабатываемое приложение предназначено для автоматизации учета, сопровождения и контроля заявок на ремонт климатической техники в сервисной организации ООО «Конди».

В соответствии с техническим заданием система должна обеспечивать выполнение следующих функциональных требований:

— создание и сохранение заявок на ремонт в базе данных;

— просмотр и изменение информации по уже зарегистрированным заявкам;

— контроль текущего состояния заявки на всех этапах выполнения работ;

— назначение специалиста, ответственного за выполнение ремонта;

— ведение комментариев и служебных заметок по каждой заявке;

— учет комплектующих и дополнительных сведений, связанных с ремонтом;

— формирование статистических данных по работе сервиса;

— контроль просроченных заявок и обращений, требующих дополнительного внимания;

— предоставление QR-кода для перехода к форме оценки качества обслуживания.

Помимо функциональных возможностей, программа должна удовлетворять следующим нефункциональным требованиям:

— надежность хранения и обработки данных;

— защита доступа к системе с учетом ролей пользователей;

— удобный и понятный пользовательский интерфейс;

— приемлемая производительность при работе со списками заявок и отчетами;

— возможность дальнейшего расширения и доработки приложения.

Отдельно стоит отметить, что при разработке было необходимо выбрать такие средства реализации, которые позволили бы создать удобное веб-приложение с поддержкой базы данных, авторизации пользователей и разграничения прав доступа.

Входными данными для программы являются:

— данные учетных записей пользователей – логин, пароль, ФИО, телефон;

— сведения, вводимые при создании заявки – тип оборудования, модель устройства, описание неисправности, данные клиента;

— информация, добавляемая в ходе обработки заявки – статус ремонта, назначенный специалист, комментарии, перечень комплектующих, отметки о завершении;

— служебные записи, связанные с запросами помощи или контролем качества.

Выходными данными являются:

— список заявок с возможностью поиска и фильтрации по статусу, типу оборудования и другим параметрам;

— карточка заявки с полной информацией о ремонте;

— системные уведомления об успешном выполнении операций или об ошибках;

— статистические сведения по количеству заявок, срокам выполнения и наиболее частым неисправностям;

— данные для контроля просроченных заявок и обращений в модуле качества;

— сгенерированный QR-код для оценки качества обслуживания.

1.2 Выбор стека технологий и инструментов разработки

Для создания данного программного продукта был выбран стек технологий на базе платформы .NET с использованием языка программирования C#. В качестве среды разработки применялась Visual Studio, так как она предоставляет удобные инструменты для написания, запуска и отладки кода, а также хорошо подходит для работы с веб-проектами на ASP.NET Core. Выбор языка C# и платформы .NET обусловлен их надежностью, современной архитектурой и большим количеством готовых средств для разработки прикладных информационных систем.

Для реализации пользовательского интерфейса использовалась технология ASP.NET Core. Данный подход позволяет строить веб-приложение на основе страниц, где логика обработки и интерфейс тесно связаны между собой. Это упрощает разработку приложений небольшого и среднего объема, делает структуру проекта более понятной и позволяет быстрее реализовать формы создания, редактирования, просмотра и удаления заявок.

В качестве системы хранения данных была выбрана Pos. Эта система управления базами данных удобна тем, что не требует отдельной настройки серверной части, легко подключается к приложению и подходит для учебных и демонстрационных проектов. Использование PostgreSQL позволило организовать надежное хранение сведений о пользователях, заявках на ремонт и комментариях без необходимости разворачивать отдельный сервер базы данных.

Для взаимодействия с базой данных применялась технология Entity Framework Core. Она обеспечивает работу с таблицами через объектные модели C#, что значительно упрощает разработку. Благодаря EF Core были реализованы

описание сущностей, связи между ними, автоматическое создание структуры базы данных и выполнение запросов без необходимости вручную писать большое количество SQL-кода.

Для организации входа в систему и разграничения доступа использовалась библиотека ASP.NET Core Identity. Она позволила реализовать безопасную аутентификацию пользователей, хранение учетных записей, распределение ролей и контроль прав доступа к различным разделам приложения. Проверка полномочий выполняется с помощью механизма ролей и атрибутов авторизации, благодаря чему разные категории пользователей – менеджер, оператор, специалист, заказчик и менеджер по качеству – получают доступ только к тем функциям, которые соответствуют их обязанностям.

Дополнительно в проекте была использована библиотека QR Coder, с помощью которой реализована генерация QR-кода для перехода к форме оценки качества обслуживания. Это расширяет функциональные возможности системы и делает приложение более удобным для конечных пользователей.

Таким образом, выбранный набор технологий позволил реализовать веб-приложение с поддержкой базы данных, авторизации, ролевой модели доступа и средствами управления заявками на ремонт. Использованный стек является современным, удобным для сопровождения и хорошо подходит для решения поставленной задачи.

1.3 Разработка базы данных

Перед проектированием базы данных были определены основные сущности предметной области, необходимые для хранения информации о пользователях, заявках на ремонт и комментариях к ним. В результате были выделены следующие сущности и их атрибуты:

user:
- Id (PK);
- Fio;
- Phone;
- Login;
- Password;
- Type;
- IsDeleted.

climate_tech_type:
- Id (PK);
- Name;
- Description.

request_status:
- Id (PK);
- Name;

Description.

repair_request:

Id (PK);
StartDate;
TypeId (FK);
Model;
ProblemDescription;
StatusId (FK);
CompletionDate;
RepairParts;
MasterId (FK);
ClientId (FK);
QualityManagerId (FK);
DeadlineExtended.

request_comment:

Id (PK);
Message;
MasterId (FK);
RequestId (FK).

Связи между сущностями:

– пользователи и заявки по назначенному специалисту (один ко многим): один пользователь может быть назначен специалистом в нескольких заявках (внешний ключ master_id в таблице requests ссылается на user_id в таблице users), при этом у каждой заявки может быть один назначенный специалист;

– пользователи и заявки по клиенту (один ко многим): один пользователь может выступать клиентом в нескольких заявках (внешний ключ client_id в таблице requests ссылается на user_id в таблице users), при этом каждая заявка может быть связана с одним клиентом;

– пользователи и заявки по менеджеру по качеству (один ко многим): один пользователь может быть назначен менеджером по качеству в нескольких заявках (внешний ключ quality_manager_id в таблице requests ссылается на user_id в таблице users), при этом у каждой заявки может быть один менеджер по качеству;

– заявки и статусы (многие к одному): каждая заявка имеет один статус (внешний ключ status_id в таблице requests ссылается на status_id в таблице request_statuses), при этом один статус может быть присвоен множеству заявок;

– заявки и типы климатической техники (многие к одному): каждая заявка относится к одному типу техники (внешний ключ type_id в таблице requests ссылается на type_id в таблице climate_tech_types), при этом один тип техники может фигурировать во множестве заявок;

– заявки и комментарии (один ко многим): одна заявка может содержать несколько комментариев (внешний ключ request_id в таблице comments ссылается на request_id в таблице requests), а каждый комментарий относится только к одной заявке;

– пользователи и комментарии (один ко многим): один пользователь может оставить несколько комментариев (внешний ключ master_id в таблице comments ссылается на user_id в таблице users), при этом каждый комментарий имеет одного автора.

ER-диаграмма системы представлена на рисунке 1.1.

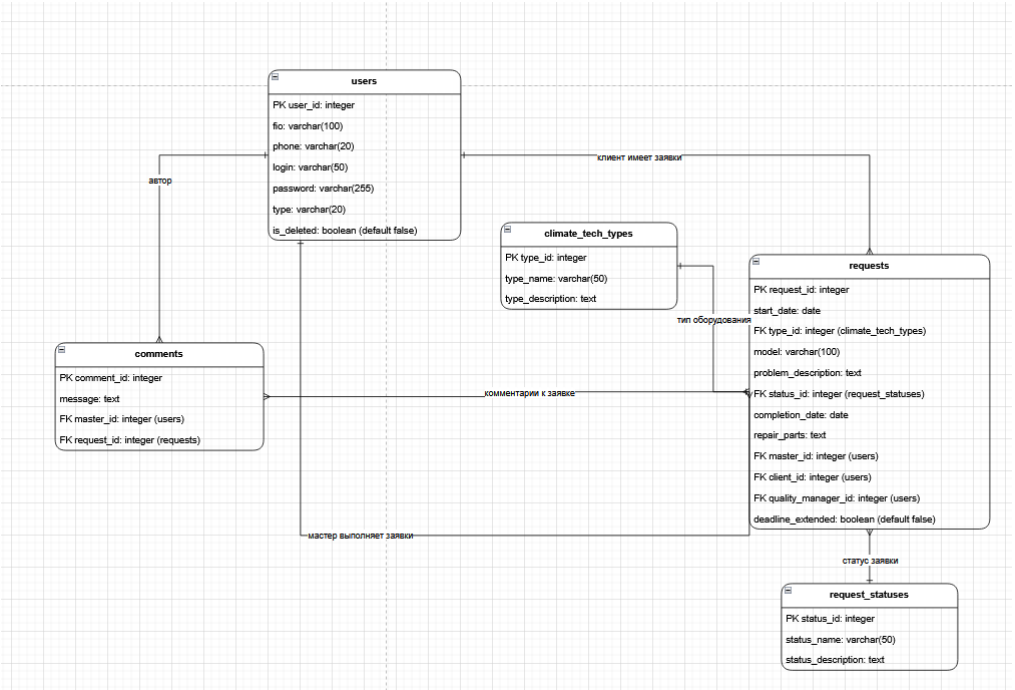

Рисунок 1.1 – ER-диаграмма

Запросы создания и заполнения базы данных представлены в приложении А.

1.4 Практическая реализация приложения

Перед разработкой программного продукта была составлена блок-схема основного алгоритма работы системы. На данной схеме отражена последовательность действий пользователя при входе в приложение, проверке корректности учетных данных, определении роли и дальнейшем выполнении доступных операций. После успешной авторизации пользователь получает доступ к главному окну системы, где может выполнять действия в соответствии со своей ролью: создавать, редактировать и просматривать заявки, добавлять комментарии, анализировать статистику и завершать работу с системой. Блок-схема позволяет наглядно представить общую логику функционирования приложения и порядок перехода между основными этапами работы.

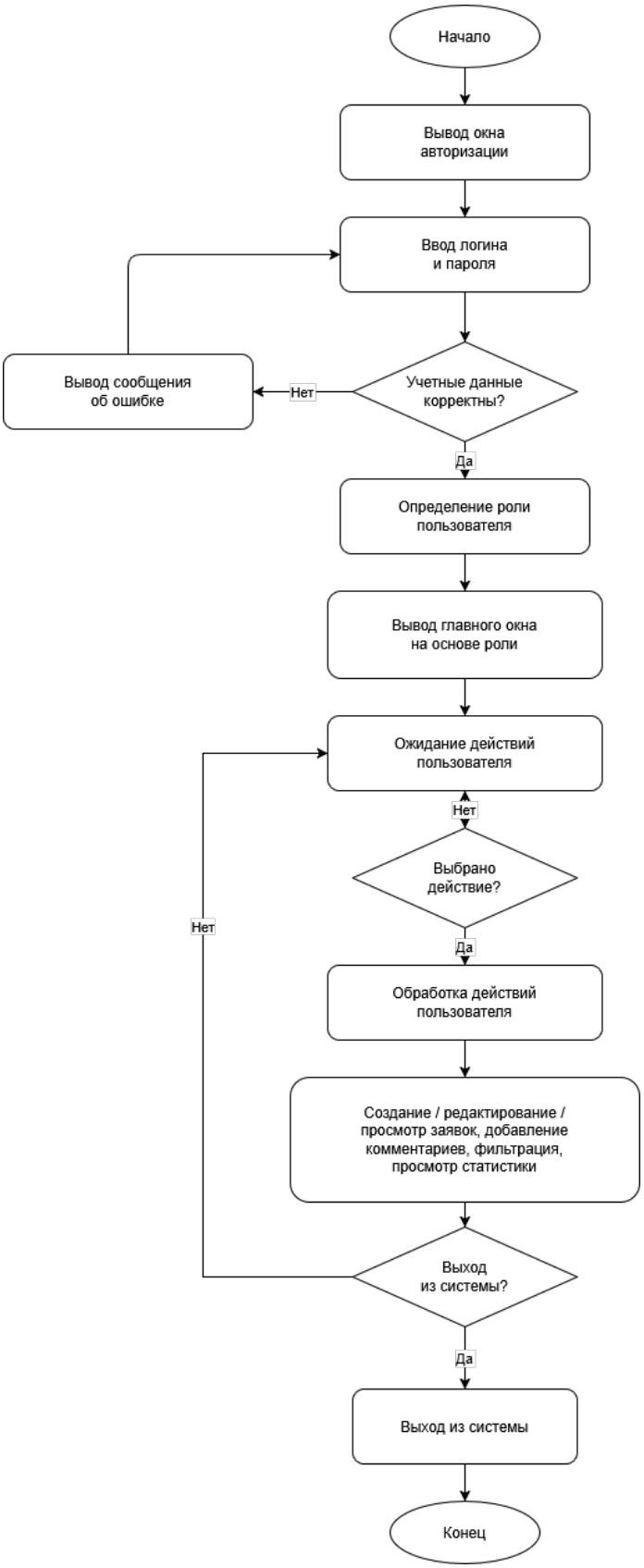

Рисунок 1.2 – Основной алгоритм работы системы

Программное обеспечение реализовано в виде веб-приложения на платформе ASP.NET Core, что позволяет разделить структуру проекта на модели данных, работу с базой данных, страницы интерфейса и служебные компоненты.

Компоненты системы:

– Model: структура базы данных представлена классами ApplicationUser, RepairRequest и RequestComment. Сущность заявки связана с пользователем как с клиентом и как с назначенным специалистом, а комментарий связан с заявкой и автором комментария;

– Data: для взаимодействия с базой данных используется AppDbContext, содержащий наборы сущностей DbSet. Связи между таблицами настраиваются в методе OnModelCreating, где также задаются правила удаления связанных данных;

– View: пользовательский интерфейс реализован с помощью Razor Pages. Страницы обеспечивают просмотр списка заявок, создание, редактирование, удаление, просмотр статистики, комментариев и других разделов системы;

– Controller: основная логика размещена в моделях страниц, например в разделах Requests, Quality, Stats, Users и Account. В них выполняются проверка прав доступа, загрузка данных, обработка действий пользователя и сохранение изменений;

– разделение по ролям: в приложении используются роли менеджера, оператора, специалиста, заказчика и менеджера по качеству. В зависимости от роли пользователю открываются только разрешённые функции;

– действия по ролям: оператор и менеджер могут создавать и изменять заявки, специалист может обновлять статус ремонта, добавлять комментарии и указывать комплектующие, заказчик имеет ограниченный доступ к просмотру, а менеджер по качеству контролирует проблемные и просроченные заявки.

Вход пользователя в систему реализован через страницу авторизации. После успешной проверки логина и пароля создаётся пользовательская сессия, а дальнейший доступ к разделам ограничивается с помощью механизма авторизации и проверки ролей.

Работа с заявками реализована в разделе Requests, который содержит основные операции:

– список заявок: загружается перечень заявок с возможностью поиска, фильтрации по статусу и типу оборудования, а также сортировки по дате создания;

– карточка заявки: отображает полную информацию по заявке, назначенного специалиста, данные клиента и комментарии;

– создание заявки: при добавлении новой заявки автоматически устанавливаются дата создания, начальный статус и срок выполнения;

– редактирование заявки: состав доступных для изменения полей зависит от роли пользователя;

– удаление заявки: удаление доступно только менеджеру.

Раздел комментариев позволяет добавлять сообщения по заявке и фиксировать внутренние замечания в процессе ремонта. Для специалистов также предусмотрена возможность отправки запроса помощи по заявке.

Раздел Model предназначен для расчёта и отображения статистических данных. В нём выводятся:

– общее количество заявок;
– количество завершённых ремонтов;
– среднее время выполнения;
– наиболее частые неисправности.

Дополнительно в приложении предусмотрены средства повышения удобства работы:

– обработка ошибок доступа и отсутствующих данных;
– вывод уведомлений об успешных действиях и ошибках;
– генерация QR-кода для перехода к форме оценки качества обслуживания.

Таким образом, практическая реализация программы охватывает все основные функции системы учёта заявок на ремонт климатического оборудования и обеспечивает работу пользователей в соответствии с их ролями и полномочиями.

1.5 Разработка веб-приложения

В результате разработки программы реализовано следующее:
– меню вкладок с таблицами;
– права доступа согласно ролям;
– авторизация по логину и паролю;
– CRUD операции;
– обработка ошибок и предупреждений при удалении данных;
– вкладка со статистикой работы отдела;
– отображение данных текущего пользователя и функция выхода из системы.

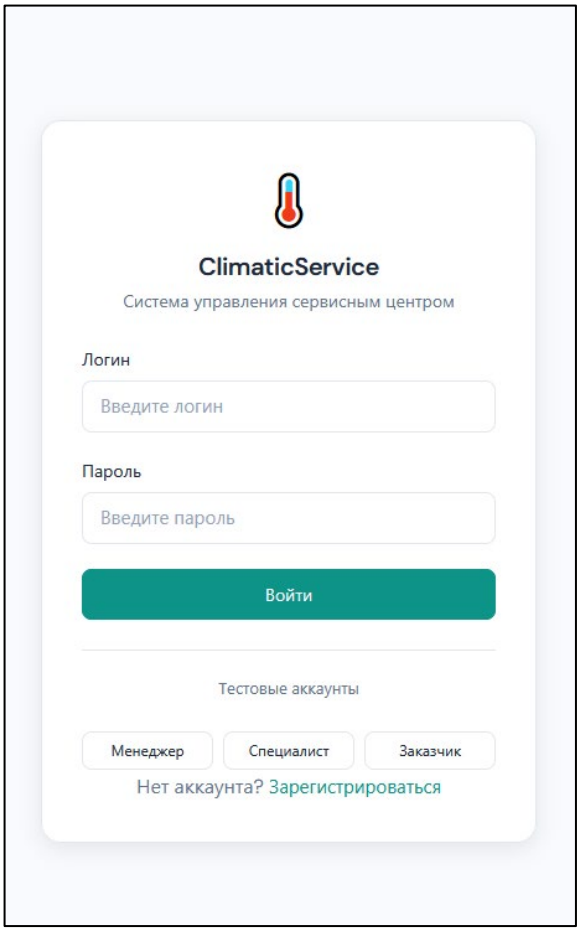

Рисунок 1.3 – Окно авторизации

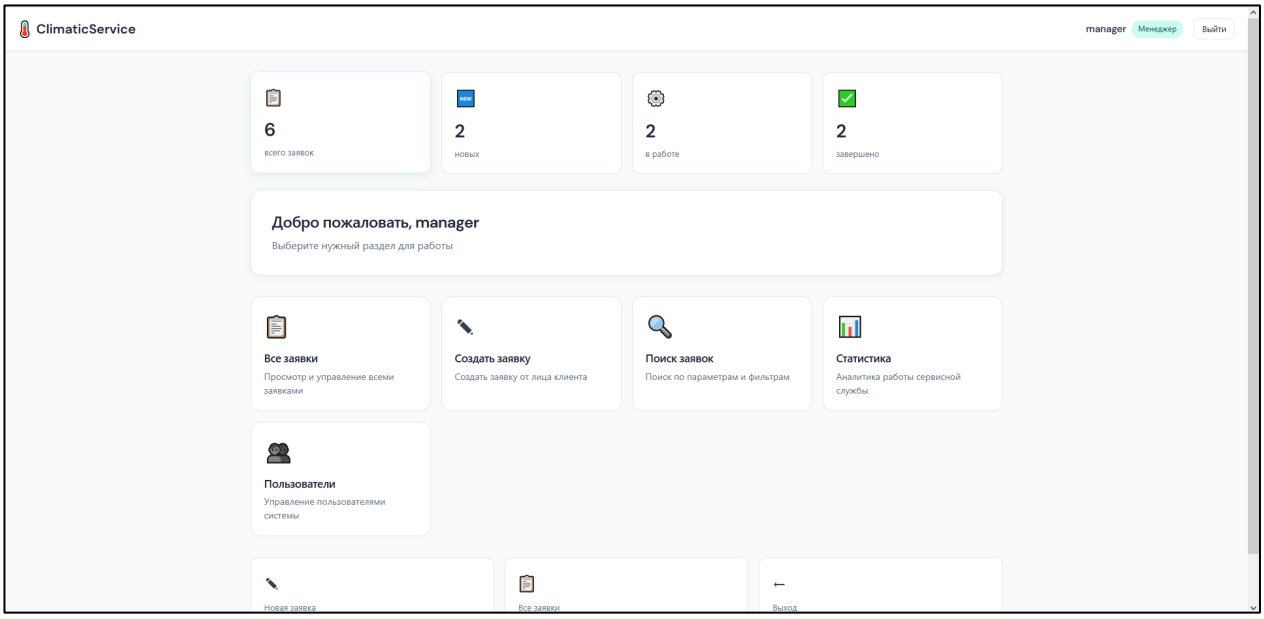

Рисунок 1.4 – Главная вкладка

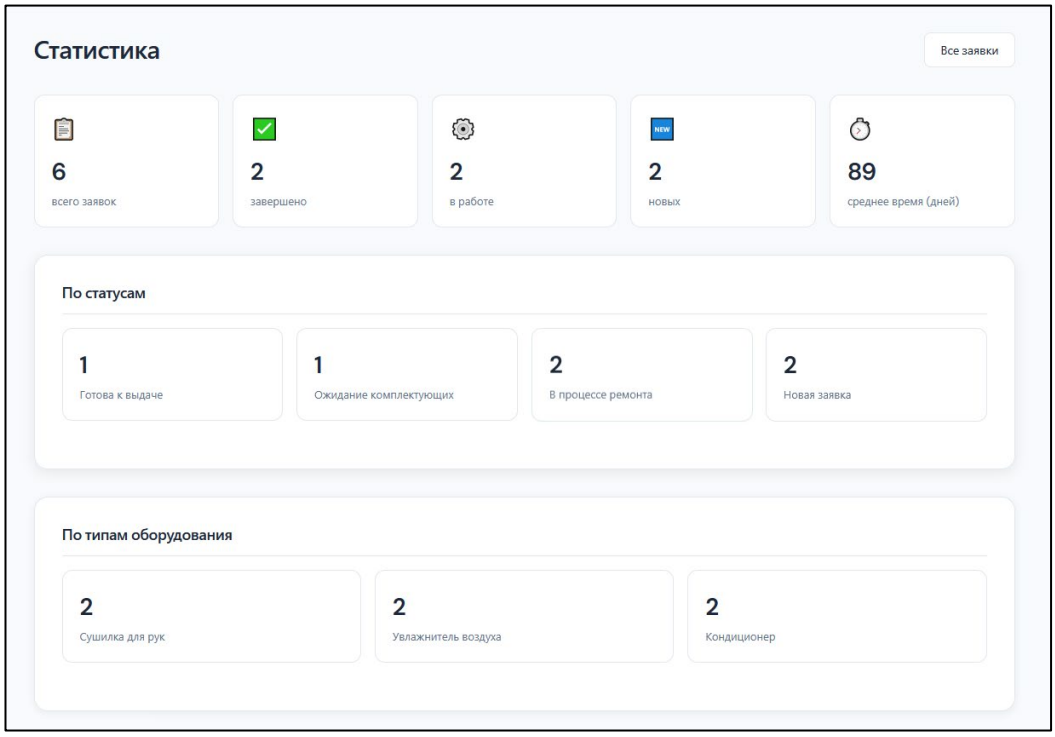

Рисунок 1.5 – Окно статистики

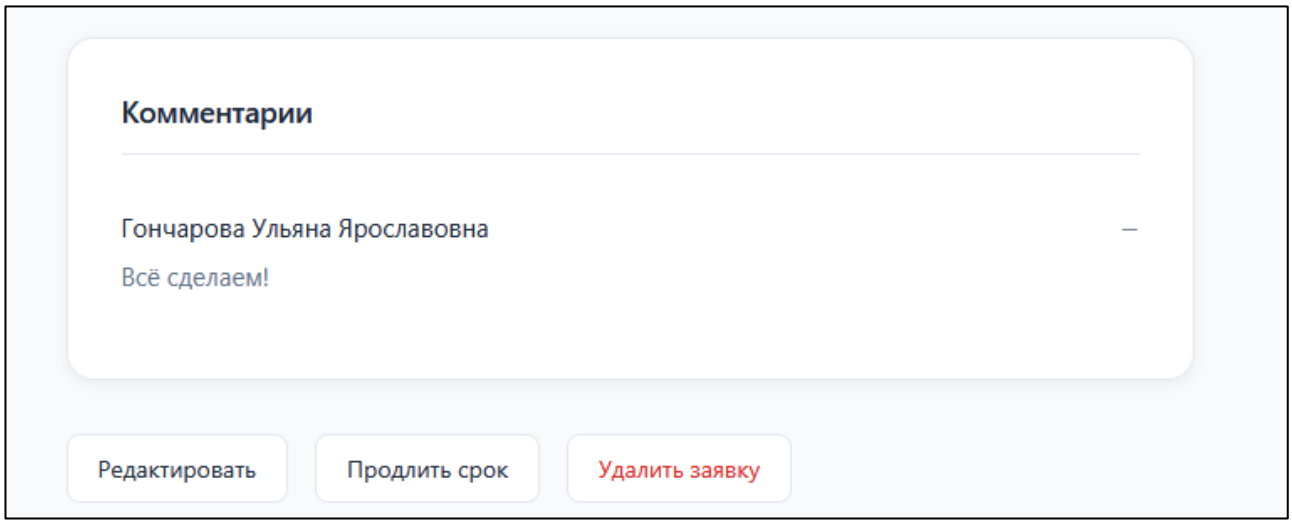

Рисунок 1.6 – Окно комментариев

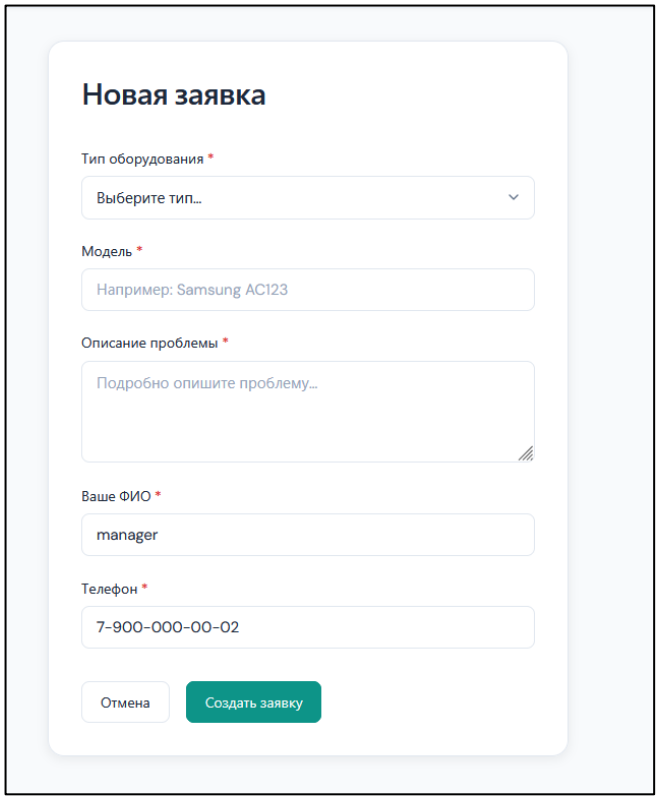

Рисунок 1.7 – Создание заявки

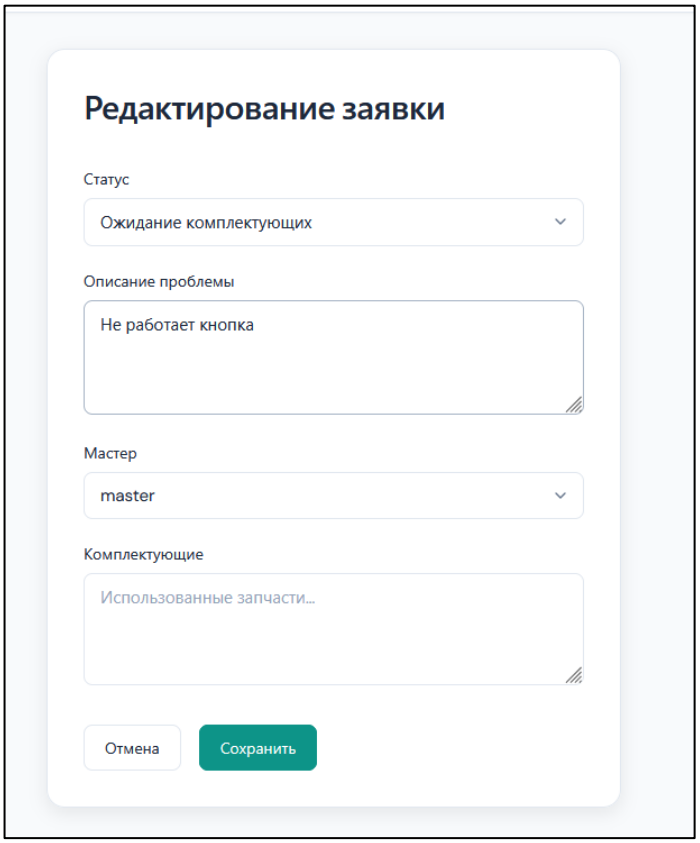

Рисунок 1.8 – Редактирование заявки

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.1.

Таблица 1.1 – Тестирование программы

№ теста	Модуль тестирования	Порядок тестирования	Ожидаемый результат	Результат теста
1	Логин	Ввод корректного пароля и логина	Вход в систему	Успешный вход
2	Логин	Ввод неправильного пароля или логина	Появление ошибки	Вывод ошибки без входа в систему
3	Ролевой доступ	Поочерёдный вход в систему под разными ролями пользователей	Успешное отображение таблиц в соответствии с ролями	У каждой роли отображается свой функционал

Таблица 1.2 – Тестирование программы

4	Работа с данными	Создать заявку	Заявка успешно создана	Успешно созданная заявка
5	Работа с данными	Удалить заявку	Заявка удалена	Успешно удаленная заявка
6	Работа с данными	Изменение заявки	Заявка изменена	Успешно измененная заявка
7	Работа с данными	Добавление комментария	Появление комментария	Не созданный комментарий к заявке
8	Работа с данными	Создание заявки с пустыми полями	Сообщение о пустых полях	Отображение ошибки о некорректности введенных данных

1.7 Качественная характеристика кода

После тестирования была выполнена оценка качества программного кода. Определение качественных характеристик кода: полнота обработки ошибочных данных включает проверки на отсутствие записей в базе данных, проверки корректности введённых данных, проверку пустых строк, проверку прав доступа по ролям, а также обработку ошибок при сохранении изменений в базе данных; однако отсутствуют централизованный механизм логирования ошибок, не все возможные исключительные ситуации обрабатываются отдельно, часть проверок реализована только на базовом уровне; таким образом, обработка ошибок реализована, но может быть дополнительно улучшена.

Автоматические тесты для проверки допустимых значений входных данных в проекте отсутствуют. Что касается средств контроля корректности входных данных, то присутствуют проверка модели, контроль заполнения обязательных полей, проверка ролей пользователей, проверка допустимости действий над заявками и комментариями, а также ограничения на доступ к отдельным функциям; однако не для всех полей реализована расширенная валидация формата и длины значений; таким образом, базовые средства контроля корректности входных данных присутствуют. Средства восстановления при сбоях оборудования в приложении отсутствуют: не реализованы автоматическое резервное копирование базы данных, журналирование с восстановлением, повторное выполнение операций при сбоях; следовательно, встроенные средства восстановления после сбоев отсутствуют. По наличию комментариев можно отметить, что структура кода и имена классов, свойств и методов в целом понятны, отдельные части кода снабжены пояснениями, однако подробное документирование реализовано не во всех файлах; таким образом, код в целом читаемый, но количество комментариев можно увеличить в наиболее важных участках.

Проверка корректности передаваемых данных выполняется: присутствуют проверка идентификаторов записей, существования заявок и пользователей, а также проверка прав на просмотр, изменение и удаление данных; итогом является то, что проверка корректности передаваемых данных реализована. Описания основных функций присутствуют: основные функции имеют понятные названия и логичное разделение по разделам приложения, например создание заявки, редактирование, просмотр списка, работа со статистикой, комментариями и авторизацией; таким образом, основные функции программы выделены и читаются достаточно понятно.

2 Дипломный проект

2.1 Анализ предметной области

Колледж предпринимательства и социального управления (КПСУ) является образовательным учреждением среднего профессионального образования. Основное направление деятельности - подготовка специалистов в области предпринимательства, экономики, права и управления. Колледж осуществляет образовательный процесс с использованием современных технологий, включая дистанционные формы обучения.

Организационная структура колледжа предпринимательства и социального управления включает следующие основные подразделения:

Администрация колледжа:

− осуществляет общее руководство деятельностью колледжа;

− координирует работу всех структурных подразделений;

− отвечает за стратегическое развитие и организацию учебного процесса.

Заочное отделение:

− осуществляет организацию учебного процесса для студентов заочной формы обучения;

− координирует проведение установочных и экзаменационных сессий;

− управляет вопросами учебно-методического обеспечения заочников.

Заочное дистанционное отделение:

− организует образовательный процесс с использованием дистанционных технологий;

− обеспечивает работу электронной информационно-образовательной среды;

− координирует взаимодействие студентов и преподавателей в онлайн-формате.

Бухгалтерия колледжа:

− осуществляет финансовую деятельность;

− осуществляет расчет и начисление заработной платы;

− отвечает за налоговую отчетность и финансовое планирование.

Преподавательский состав колледжа:

− включает штатных преподавателей и наемных;

− обеспечивает реализацию образовательных программ;

− проводит аудиторные и дистанционные занятия.

2.2 Разработка дополнительного функционала

В личном кабинете пользователя была реализована функция изменения пароля. Данная функция позволяет пользователю самостоятельно обновлять свой пароль для учётной записи в целях безопасности или в случае утери старого. Для этого в соответствующем разделе настроек безопасности была добавлена форма, состоящая из трёх полей ввода: для указания текущего пароля, для ввода нового пароля и для его подтверждения.

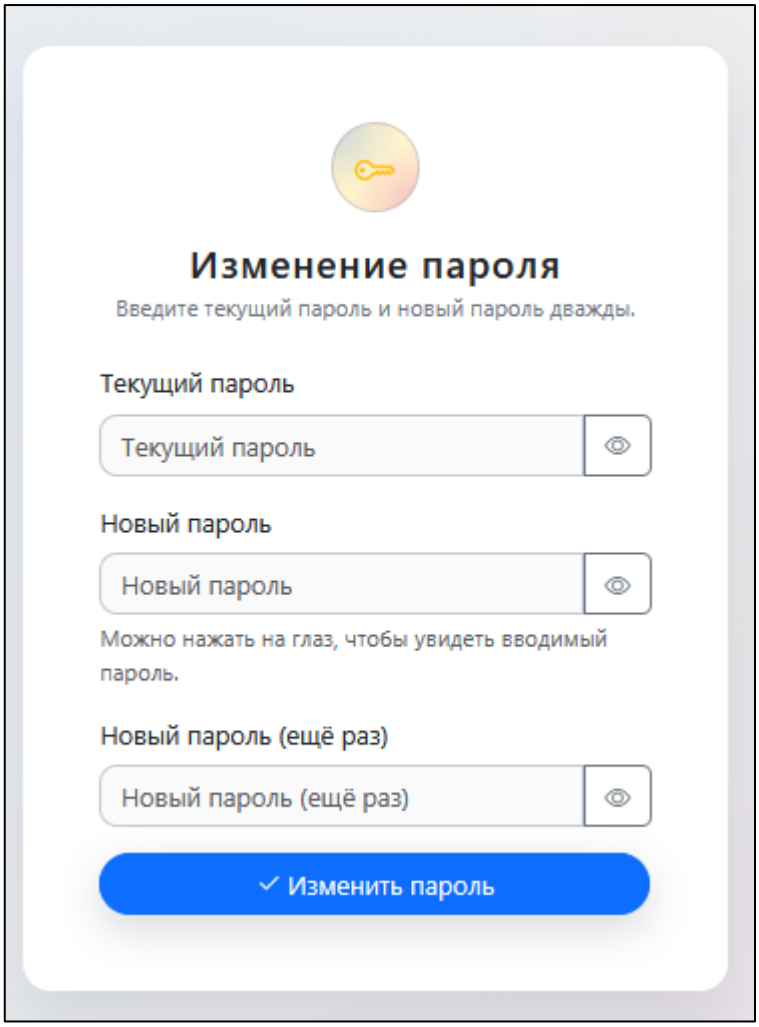

Рисунок 2.1 – Страница смены пароля

Интерфейс изменения пароля, представленный на изображении, представляет собой типовую форму, состоящую из трёх основных полей ввода. В верхней части формы находится заголовок «Изменение пароля» и краткая инструкция, предлагающая пользователю ввести текущий пароль, а затем дважды указать новый пароль для подтверждения.

Первое поле формы предназначено для ввода текущего пароля пользователя.

Оно помечено меткой «Текущий пароль» и, согласно описанию, может быть снабжено интерактивной иконкой в виде глаза. Нажатие на эту иконку позволяет временно отобразить введённые символы, что помогает пользователю избежать ошибок при наборе.

Второе поле формы служит для ввода нового пароля. Оно имеет метку «Новый пароль» и также, вероятно, оснащено функцией показа символов через ту же иконку глаза. Это поле является ключевым, так как здесь устанавливается новая секретная комбинация.

Третье поле, обозначенное как «Новый пароль (ещё раз)», предназначено для повторного ввода только что указанной новой комбинации. Его основная функция — верификация: система сравнивает содержимое второго и третьего полей. Если они не совпадают, пользователь получит уведомление об ошибке, что предотвратит установку пароля, отличного от задуманного из-за опечатки.

В нижней части формы располагается кнопка с текстом «Изменить пароль». Нажатие на эту кнопку инициирует процесс проверки и обработки данных: система сначала сверяет введённый текущий пароль с сохранённым в базе данных, затем проверяет новый пароль на соответствие политикам безопасности (сложность, длина, уникальность) и, наконец, сравнивает два ввода нового пароля.

Только при успешном прохождении всех проверок новый пароль будет сохранён, и пользователь получит подтверждение об успешном изменении. Вся эта логика, включая валидацию и обработку ошибок, реализуется в функции изменения пароля, которая является неотъемлемой частью модуля безопасности и управления учётными данными пользователя.

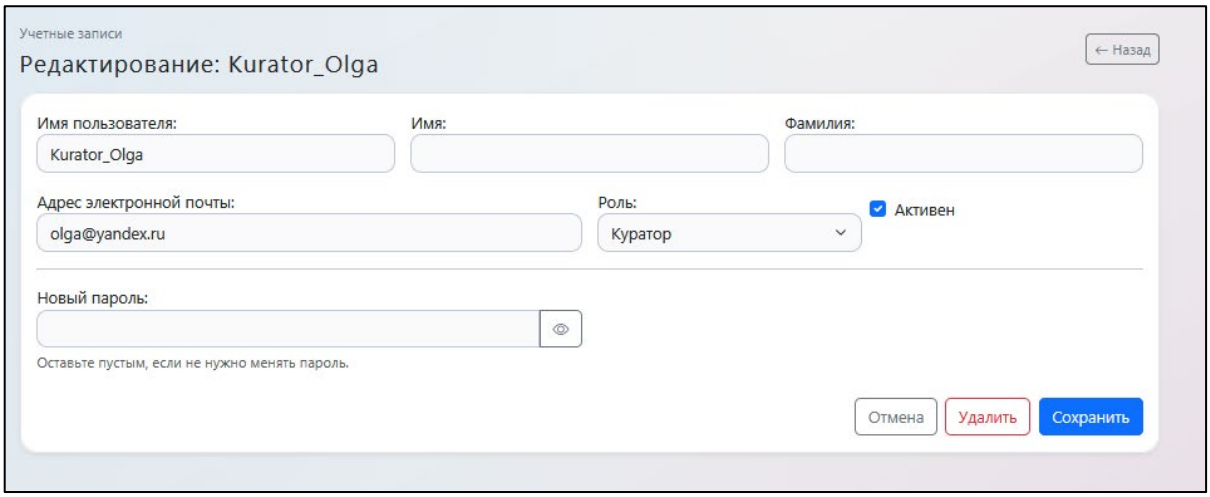

Рисунок 2.2 – Страница редактирования пользователя

Интерфейс управления учетными записями, представленный на изображении, представляет собой типовую форму редактирования данных пользователя, доступную только администратору системы. В верхней части формы находится заголовок «Редактирование: Логин пользователя», который четко

указывает, что в данный момент происходит редактирование конкретной учетной записи, а также отображает имя редактируемого пользователя.

Первое поле формы предназначено для отображения имени пользователя (логина). Оно помечено меткой «Имя пользователя» и содержит значение «Логин». Данное поле, скорее всего, является недоступным для редактирования, так как логин обычно служит уникальным идентификатором учетной записи и не подлежит изменению.

Далее следуют два поля для ввода персональных данных пользователя: «Имя» и «Фамилия». Эти поля предназначены для заполнения или корректировки личных данных владельца учетной записи. Они могут быть как обязательными, так и опциональными в зависимости от настроек системы, но в любом случае позволяют администратору поддерживать актуальность информации о пользователях.

Четвертое поле формы служит для указания адреса электронной почты пользователя. Оно помечено меткой «Адрес электронной почты» и содержит значение «Почта». Электронная почта является важным контактным данным, которые могут использоваться для уведомлений, восстановления доступа или других коммуникаций с пользователем.

Пятое поле, обозначенное как «Роль», представляет собой элемент выбора роли пользователя в системе. В данном случае выбрано значение «Куратор». Это поле критически важно, так как именно через него администратор может изменять права доступа пользователя, назначая ему соответствующую роль, например «Бухгалтер», «Преподователь». Изменение роли напрямую влияет на набор функций и разделов, доступных данному пользователю в системе.

Шестой блок формы посвящен изменению пароля. Он содержит поле «Новый пароль» и краткую инструкцию «Оставьте пустым, если не нужно менять пароль». Это означает, что администратор может установить новый пароль для пользователя, например, при создании учетной записи или в случае, если пользователь забыл свои данные доступа. Поле ввода пароля может быть снабжено функцией отображения символов для удобства ввода сложных комбинаций.

В нижней части формы располагаются три кнопки управления: «Отмена», «Удалить» и «Сохранить». Кнопка «Отмена» позволяет выйти из режима редактирования без сохранения изменений. Кнопка «Удалить» предназначена для удаления учетной записи пользователя из системы, что должно сопровождаться дополнительным подтверждением для предотвращения случайных действий. Кнопка «Сохранить» инициирует процесс проверки и сохранения всех внесенных изменений: система проверяет корректность заполненных полей, уникальность данных (например, электронной почты), а при наличии нового пароля применяет политики безопасности к нему, после чего обновляет информацию о пользователе в базе данных.

Таким образом, созданный интерфейс предоставляет администратору полный контроль над управлением учетными записями пользователей, позволяя редактировать личные данные, изменять роли и права доступа, а также при

необходимости сбрасывать или устанавливать новые пароли, что является неотъемлемой частью модуля администрирования и безопасности системы.

В веб-приложении реализован модуль «Календарь», предназначенный для визуального отображения учебной или рабочей нагрузки пользователя в разрезе дней, недель и месяцев. Данный инструмент особенно полезен для преподавателей, студентов или сотрудников, которым необходимо планировать своё время, отслеживать расписание занятий, встреч или событий.

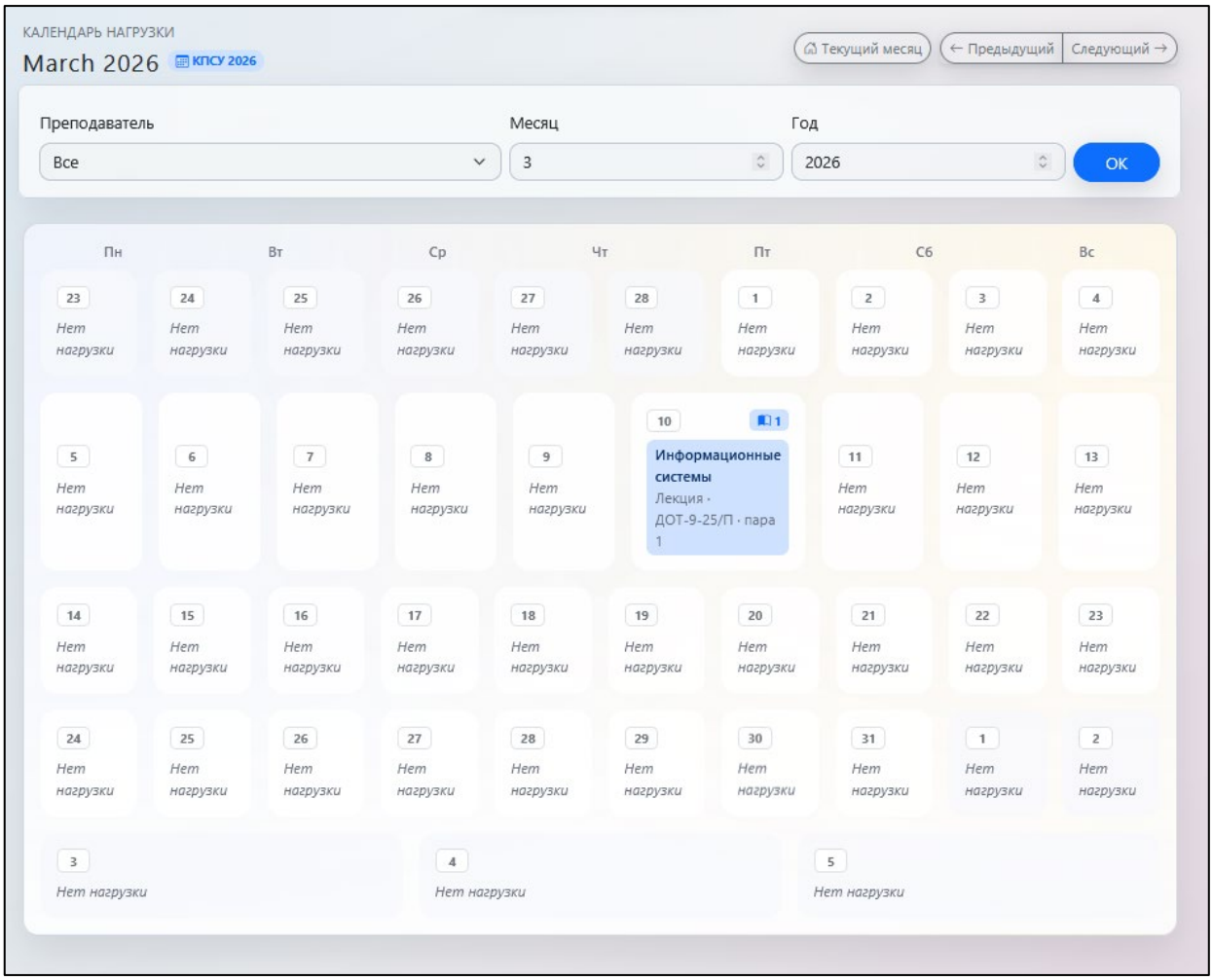

Рисунок 2.3 – Календарь нагрузки в веб-приложении

Внутри блока дня перечислены запланированные мероприятия: например, лекции, практические занятия, лабораторные работы. Каждое мероприятие сопровождается подробной информацией - указывается название дисциплины, тип занятия, группа или поток, а также номер пары или временной интервал. Если на определённый день нагрузка отсутствует, в блоке отображается соответствующая пометка - «Нет нагрузки». Для удобства восприятия разные типы занятий могут быть выделены цветом или значками. Календарь поддерживает переключение между режимами отображения - «Месяц», «Неделя» или «День», что позволяет

гибко адаптировать визуализацию под текущие задачи. Все данные синхронизируются с сервером в реальном времени, поэтому любые изменения в расписании (перенос или отмена занятий) сразу же отражаются в интерфейсе.

Заключение

В ходе учебной практики было разработано веб-приложение для автоматизации учета и сопровождения заявок на ремонт климатической техники. В процессе выполнения работы была проанализирована предметная область, изучены требования к программному продукту, выбраны подходящие технологии разработки и спроектирована структура базы данных. На основе этого было реализовано приложение, обеспечивающее авторизацию пользователей, разграничение прав доступа по ролям, создание, просмотр, редактирование и контроль заявок, а также ведение комментариев, статистики и дополнительных сервисных функций.

Разработанное решение выполнено на платформе ASP.NET Core с использованием языка C#, библиотеки Entity Framework Core и базы данных PostgreSQL. В результате была создана работоспособная информационная система, соответствующая поставленным требованиям. Итогом практики стало закрепление навыков проектирования базы данных, разработки веб-приложений, организации пользовательского интерфейса и документирования программного продукта. Поставленная цель учебной практики была достигнута в полном объеме.

Библиография

1 Учёт рабочего времени и онлайн табелирование: современные решения / Под ред. И.В. Новикова. – М.: Издательство «Технопроект», 2024. – 290 с.

2 Разработка веб-приложений для управления персоналом / А.Н. Смирнов, Е.В. Ковалёва. – СПб.: Питер, 2023. – 320 с.

3 Автоматизация расчета заработной платы в образовательных учреждениях / В.Д. Иванов. – М.: Инфра-М, 2022. – 245 с.

4 Основы веб-технологий для бизнес-приложений / Д.А. Морозов. – М.: Интернет-университет информационных технологий, 2023. – 280 с.

5 JavaScript и современные фреймворки для создания интерактивных интерфейсов / К.П. Васильев. – СПб.: Питер, 2022. – 416 с.

6 Базы данных и SQL: проектирование и применение / Л.М. Федорова. – М.: ДМК Пресс, 2023. – 264 с.

7 Безопасность web-приложений: теория и практика / С.И. Павлов. – М.: Лаборатория знаний, 2022. – 183 с.

8 HTML5 и CSS3 в разработке современных веб-приложений / М.А. Семенова. – М.: Эксмо, 2023. – 320 с.

Приложение А
(справочная)
Листинг программы

Запросы на создание и заполнение базы данных

```
CREATE SEQUENCE public.climate_tech_types_type_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.climate_tech_types_type_id_seq OWNER TO
postgres;

SET default_tablespace = '';

SET default_table_access_method = heap;

--
-- TOC entry 223 (class 1259 OID 33054)
-- Name: climate_tech_types; Type: TABLE; Schema: public; Owner:
postgres
--

CREATE TABLE public.climate_tech_types (
    type_id                  integer                  DEFAULT
nextval('public.climate_tech_types_type_id_seq'::regclass) NOT NULL,
    type_name character varying(50) NOT NULL,
    type_description text
);


ALTER TABLE public.climate_tech_types OWNER TO postgres;

--
-- TOC entry 221 (class 1259 OID 33043)
-- Name: comments_comment_id_seq; Type: SEQUENCE; Schema:
public; Owner: postgres
--

CREATE SEQUENCE public.comments_comment_id_seq
    START WITH 4
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
```

```
ALTER SEQUENCE public.comments_comment_id_seq OWNER TO postgres;

--
-- TOC entry 226 (class 1259 OID 33109)
-- Name: comments; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.comments (
    comment_id                 integer                 DEFAULT
nextval('public.comments_comment_id_seq'::regclass) NOT NULL,
    message text NOT NULL,
    master_id integer NOT NULL,
    request_id integer NOT NULL
);


ALTER TABLE public.comments OWNER TO postgres;

--
-- TOC entry 219 (class 1259 OID 33041)
-- Name: request_statuses_status_id_seq; Type: SEQUENCE; Schema:
public; Owner: postgres
--

CREATE SEQUENCE public.request_statuses_status_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER  SEQUENCE  public.request_statuses_status_id_seq  OWNER  TO
postgres;

--
-- TOC entry 224 (class 1259 OID 33064)
-- Name: request_statuses; Type: TABLE; Schema: public; Owner:
postgres
--

CREATE TABLE public.request_statuses (
    status_id                  integer                 DEFAULT
nextval('public.request_statuses_status_id_seq'::regclass) NOT NULL,
    status_name character varying(50) NOT NULL,
    status_description text
);


ALTER TABLE public.request_statuses OWNER TO postgres;

--
-- TOC entry 220 (class 1259 OID 33042)
```

```
--  Name:  requests_request_id_seq;  Type:  SEQUENCE;  Schema:
public; Owner: postgres
--

CREATE SEQUENCE public.requests_request_id_seq
    START WITH 6
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;


ALTER SEQUENCE public.requests_request_id_seq OWNER TO postgres;

--
-- TOC entry 225 (class 1259 OID 33074)
-- Name: requests; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.requests (
    request_id                 integer                 DEFAULT
nextval('public.requests_request_id_seq'::regclass) NOT NULL,
    start_date date NOT NULL,
    type_id integer NOT NULL,
    model character varying(100) NOT NULL,
    problem_description text NOT NULL,
    status_id integer NOT NULL,
    completion_date date,
    repair_parts text,
    master_id integer,
    client_id integer NOT NULL,
    quality_manager_id integer,
    deadline_extended boolean DEFAULT false,
    CONSTRAINT check_dates CHECK (((completion_date IS NULL) OR
(completion_date >= start_date)))
);


ALTER TABLE public.requests OWNER TO postgres;

--
-- TOC entry 217 (class 1259 OID 33039)
--  Name:  users_user_id_seq;  Type:  SEQUENCE;  Schema:  public;
Owner: postgres
--

CREATE SEQUENCE public.users_user_id_seq
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;
```

```
ALTER SEQUENCE public.users_user_id_seq OWNER TO postgres;

--
-- TOC entry 222 (class 1259 OID 33044)
-- Name: users; Type: TABLE; Schema: public; Owner: postgres
--

CREATE TABLE public.users (
    user_id                  integer                  DEFAULT
nextval('public.users_user_id_seq'::regclass) NOT NULL,
    fio character varying(100) NOT NULL,
    phone character varying(20) NOT NULL,
    login character varying(50) NOT NULL,
    password character varying(255) NOT NULL,
    type character varying(20) NOT NULL,
    is_deleted boolean DEFAULT false,
    CONSTRAINT users_type_check CHECK (((type)::text = ANY
((ARRAY['Менеджер'::character varying, 'Специалист'::character
varying, 'Оператор'::character varying, 'Заказчик'::character
varying, 'Менеджер по качеству'::character varying])::text[])))
);


ALTER TABLE public.users OWNER TO postgres;

--
-- TOC entry 4850 (class 0 OID 33054)
-- Dependencies: 223
-- Data for Name: climate_tech_types; Type: TABLE DATA; Schema:
public; Owner: postgres
--

INSERT INTO public.climate_tech_types VALUES (1, 'Кондиционер',
NULL);
INSERT INTO public.climate_tech_types VALUES (2, 'Увлажнитель
воздуха', NULL);
INSERT INTO public.climate_tech_types VALUES (3, 'Сушилка для
рук', NULL);


--
-- TOC entry 4853 (class 0 OID 33109)
-- Dependencies: 226
-- Data for Name: comments; Type: TABLE DATA; Schema: public;
Owner: postgres
--

INSERT INTO public.comments VALUES (1, 'Всё сделаем!', 2, 1);
INSERT INTO public.comments VALUES (2, 'Всё сделаем!', 3, 2);
INSERT INTO public.comments VALUES (3, 'Починим в момент.', 3,
3);
```

```
--
-- TOC entry 4851 (class 0 OID 33064)
-- Dependencies: 224
-- Data for Name: request_statuses; Type: TABLE DATA; Schema: public; Owner: postgres
--

INSERT INTO public.request_statuses VALUES (1, 'Новая заявка', NULL);
INSERT INTO public.request_statuses VALUES (2, 'В процессе ремонта', NULL);
INSERT INTO public.request_statuses VALUES (3, 'Готова к выдаче', NULL);
INSERT INTO public.request_statuses VALUES (4, 'Ожидание комплектующих', NULL);
INSERT INTO public.request_statuses VALUES (5, 'Завершена', NULL);
INSERT INTO public.request_statuses VALUES (6, 'Просрочена', NULL);


--
-- TOC entry 4852 (class 0 OID 33074)
-- Dependencies: 225
-- Data for Name: requests; Type: TABLE DATA; Schema: public; Owner: postgres
--

INSERT INTO public.requests VALUES (1, '2023-06-06', 1, 'TCL TAC-12CHSA/TPG-W белый', 'Не охлаждает воздух', 2, NULL, NULL, 2, 7, NULL, false);
INSERT INTO public.requests VALUES (2, '2023-05-05', 1, 'Electrolux EACS/I-09HAT/N3_21Y белый', 'Выключается сам по себе', 2, NULL, NULL, 3, 8, NULL, false);
INSERT INTO public.requests VALUES (3, '2022-07-07', 2, 'Xiaomi Smart Humidifier 2', 'Пар имеет неприятный запах', 3, '2023-01-01', NULL, 3, 9, NULL, false);
INSERT INTO public.requests VALUES (4, '2023-08-02', 2, 'Polaris PUH 2300 WIFI IQ Home', 'Увлажнитель воздуха продолжает работать при предельном снижении уровня воды', 1, NULL, NULL, NULL, 8, NULL, false);
INSERT INTO public.requests VALUES (5, '2023-08-02', 3, 'Ballu BAHD-1250', 'Не работает', 1, NULL, NULL, NULL, 9, NULL, false);


--
-- TOC entry 4849 (class 0 OID 33044)
-- Dependencies: 222
-- Data for Name: users; Type: TABLE DATA; Schema: public; Owner: postgres
--
```

```
INSERT INTO public.users VALUES (1, 'Широков Василий Матвеевич', '89210563128', 'login1', 'pass1', 'Менеджер', false);
INSERT INTO public.users VALUES (2, 'Кудрявцева Ева Ивановна', '89535078985', 'login2', 'pass2', 'Специалист', false);
INSERT INTO public.users VALUES (3, 'Гончарова Ульяна Ярославовна', '89210673849', 'login3', 'pass3', 'Специалист', false);
INSERT INTO public.users VALUES (4, 'Гусева Виктория Данииловна', '89990563748', 'login4', 'pass4', 'Оператор', false);
INSERT INTO public.users VALUES (5, 'Баранов Артём Юрьевич', '89994563847', 'login5', 'pass5', 'Оператор', false);
INSERT INTO public.users VALUES (6, 'Овчинников Фёдор Никитич', '89219567849', 'login6', 'pass6', 'Заказчик', false);
INSERT INTO public.users VALUES (7, 'Петров Никита Артёмович', '89219567841', 'login7', 'pass7', 'Заказчик', false);
INSERT INTO public.users VALUES (8, 'Ковалева Софья Владимировна', '89219567842', 'login8', 'pass8', 'Заказчик', false);
INSERT INTO public.users VALUES (9, 'Кузнецов Сергей Матвеевич', '89219567843', 'login9', 'pass9', 'Заказчик', false);
INSERT INTO public.users VALUES (10, 'Беспалова Екатерина Даниэльевна', '89219567844', 'login10', 'pass10', 'Специалист', false);
```